Page No :

Name: Nilchil Raj
Roll No: 23010095
Github: https://github.com/rajnilchil254/cloud-computing-assignment

1.) What I learnt -

(i) Knowledge distillation:- In sequence-level distillation the student learns only from the teachers outputs. The teacher, Phi3.5 (3.8B parameters), labelled 3,000 commands once, A 70M-parameter Pythia then learnt to produce the same JSON from 1,500 of those examples. It reached 100% valid JSON and 58% full-frame accuracy, against the teacher's 58%; the untrained Pythia scored 0%. Logit-based distillation was not possible, because the two models use different tokenizers and Ollama returns only text, not probabilities.

(ii) Data pipelines:- sharding by line number mod SHARD_COUNT gives balanced shards with no gaps no duplicates, and the workers need no coordination. Each worker writes its own object and saves checkpoints, so a restarted pod simply resumes.

(iii) Containers and Kubernetes:- The model weights are not in the image (0.45 GB). Each Pod downloads them from Secure FS when it starts. DNS made the readiness probe important: while a pod was loading (about 12 s), the service had no ready endpoints, so no traffic reached it.

(iv) Networking & Allocating subnets longest prefix keeps every block aligned. A stateless NACL needs separate rules for deploy traffic, and the deny rule for 198.51.100.0/24 must have a lower number than the allow rule.

(v) CI/CD: every number in my report comes from a file produced by a workflow run. The quality gate automatically rejected the untrained model (0% validity, below the 95% threshold.

2. Challenges and solutions

Tesseract inference was slow on CPU, so, I used 12 threads across six clusters. generation finished in about 49 minutes invalid JSON was caught by extract notes after the response - which I fixed by stopping generation at the first newline Under a 2 vCPU / 2-GiB limit, CPU-only PyTorch handled sequential requests with 40 ms median latency and 2 MiB peak memory. For server updates I configured read limits, checks, max surge, and a preStop delay. GitHub download failures were handled with retries and fallback sources.

3. Findings

The teacher scored 58% full-grade accuracy, the untrained student 0%, student V1 50%, and V2 51%, I trained it because V2 produced as invalid output. cleaning retained.

Page No :

2757 of 3,000 records, with two replicas, the service achieved 850 ms median latency, 1,000 ms p95 latency, 4.68 requests/second and no load-test errors. Cold start took about 14 seconds. and the HPA scaled replicas from one to two under load. Estimated cost was 0.49 per 1000 requests. and the 99% success SLO was met.

4.) What could be improved

I could improve the better prompt. filter incorrect answers, use more training data. or a larger model, and enforce the json schema during decoding. A real multi-node cluster could test fault tolerance better. caching model weights and using int8 quantization with batching could reduce startup time and improve throughput. I would also add tier-based security groups. and a read-only storage identity.